



Rapport de projet

Innovation et Planification

Réalisé par:

- Rajaa LEBNAITI

Encadré par:

- Prof Ghazdali Abdelghani

Année Universitaire: 2024/2025

Détection d'Émotions des Étudiants lors des Cours en Ligne

1. Introduction

Contexte du projet

Dans un contexte où les interactions à distance et les environnements virtuels prennent une place croissante dans les domaines éducatifs et professionnels, il devient essentiel de mieux comprendre les émotions et le comportement des participants. La capacité à analyser les émotions en temps réel, notamment lors de cours ou de réunions en ligne, offre une opportunité unique pour améliorer l'expérience des utilisateurs, renforcer leur engagement, et détecter des situations problématiques telles que la distraction ou la fatigue.

Les technologies de reconnaissance faciale et d'analyse des émotions basées sur l'intelligence artificielle jouent un rôle clé dans ce domaine. Cependant, plusieurs défis subsistent, notamment en ce qui concerne la précision de ces modèles, leur capacité à gérer des données en temps réel, ainsi que l'intégration de ces technologies dans des interfaces conviviales et accessibles.

Ce projet vise à répondre à ces problématiques en combinant des outils d'intelligence artificielle, des bases de données adaptées et une interface interactive pour permettre une meilleure compréhension des émotions des utilisateurs.

Objectifs du projet

Les objectifs principaux de ce projet sont les suivants :

1. **Développer un système de reconnaissance faciale** : Identifier les participants à l'aide d'un modèle de reconnaissance basé sur des encodages de visages, tout en différenciant les visages connus et inconnus.
2. **Analyser les émotions en temps réel** : Entraîner un modèle d'apprentissage profond capable de détecter et de classer les émotions faciales parmi des catégories prédéfinies (ex. *heureux*, *triste*, *neutre*, *intéressé*).
3. **Concevoir une interface utilisateur interactive** : Offrir une visualisation en temps réel des émotions détectées, à l'échelle individuelle et collective, avec des graphiques et des alertes visuelles pour un suivi efficace.

4. **Évaluer et améliorer le système** : Analyser les performances des modèles et identifier les axes d'amélioration, comme l'enrichissement des données ou l'optimisation des hyperparamètres.

Méthodologie générale

Le projet se déroule en plusieurs étapes structurées :

1. **Création de la base de données** :
 - Collecter un ensemble d'images pour la reconnaissance faciale et l'analyse des émotions.
 - Prétraiter ces données pour garantir leur compatibilité avec les modèles d'apprentissage automatique (redimensionnement, normalisation, etc.).
2. **Entraînement des modèles** :
 - Développer et entraîner un modèle pour la reconnaissance faciale en utilisant des technologies comme **face_recognition**.
 - Construire et entraîner un modèle de détection des émotions en utilisant des architectures CNN optimisées pour les images.
3. **Développement de l'interface utilisateur** :
 - Concevoir une application interactive avec Streamlit pour afficher les résultats des analyses en temps réel.
 - Intégrer des visualisations, telles que des graphiques d'évolution des émotions et des alertes visuelles.
4. **Tests et validation** :
 - Évaluer la performance des modèles sur des jeux de données de test.
 - Identifier les limites du système et proposer des améliorations.

Cette méthodologie garantit une approche cohérente et progressive pour atteindre les objectifs fixés tout en répondant aux défis posés par l'analyse des émotions et la reconnaissance faciale.

2. Revue de la littérature

La reconnaissance des émotions faciales chez les étudiants est un domaine de recherche en plein essor dans les environnements d'apprentissage. Cette technologie vise à analyser les émotions exprimées par les étudiants pour améliorer l'efficacité de l'enseignement et optimiser les interactions pédagogiques. Selon l'article "Deep Learning Based Student Emotion Recognition from Facial Expressions in Classrooms", plusieurs méthodes ont été proposées pour répondre aux défis de la détection et de la classification des émotions dans les salles de classe. Les approches traditionnelles reposaient sur des techniques comme les modèles géométriques et les algorithmes de reconnaissance basés sur des caractéristiques locales, tels que le modèle de forme active (ASM) et les motifs binaires locaux (LBP). Cependant, ces méthodes présentaient des limites en termes de précision et de capacité à traiter des images dans des conditions complexes, comme les angles de vision variés ou les variations d'expression faciale.

Les avancées récentes en apprentissage profond, en particulier les réseaux de neurones convolutifs (CNN), ont permis d'améliorer considérablement la précision de la reconnaissance des émotions. Des études, comme celle de Tang et al. (2018), ont montré que l'entraînement sur des bases de données comme FER2013 ou CK+ et l'utilisation de techniques de prétraitement, telles que les filtres de Gabor et la normalisation des visages, permettent de mieux gérer les complexités des environnements réels. Ces systèmes combinent souvent des caméras haute résolution avec des modèles d'apprentissage profond pour détecter les émotions comme la joie, la tristesse, ou la neutralité, fournissant ainsi un retour en temps réel sur l'état émotionnel des étudiants.

En outre, les approches basées sur le transfert d'apprentissage, comme l'utilisation de VGG16 pré-entraîné, ont démontré leur efficacité pour compenser le manque de grandes bases de données spécifiques. Ces modèles permettent une classification précise des émotions tout en réduisant le besoin d'entraînement exhaustif. Ces avancées mettent en lumière l'importance de la reconnaissance des émotions comme outil pour enrichir l'expérience d'apprentissage et favoriser une pédagogie centrée sur les besoins des étudiants.

3. Conception et développement de la base de données

- **Collection des données :**

La base de données utilisée pour ce projet combine deux ensembles de données existants : **AffectNet** et **Driver Drowsiness Dataset**, qui ont été soigneusement sélectionnés et transformés pour répondre aux besoins de la reconnaissance faciale et de l'analyse des émotions.

Collection des données

1. AffectNet

AffectNet est une base de données de référence largement utilisée pour l'analyse des émotions faciales. Elle se compose des catégories d'émotions suivantes :

- **Neutralité** : 5 000 images correspondant à des expressions neutres.
- **Enthousiasme** (*happy*) : 5 000 images représentant des visages souriants ou heureux.
- **Pas d'envie** (*sad*) : 5 000 images exprimant la tristesse ou le désintérêt.
- **Surprise** : 5 000 images montrant des expressions de surprise.
- **Mépris** (*contempt*) : 5000 images exprimant un mélange d'arrogance ou de désapprobation.

Cette base de données fournit une couverture complète des émotions fondamentales avec une bonne diversité en termes de poses et de luminosité.

2. Driver Drowsiness Dataset

Ce dataset est conçu pour détecter des états spécifiques liés à la fatigue au volant. Il est constitué de :

- **Fatigue** (*yawn + closed eyes*) : 1 243 images issues de 726 images de bâillements (*yawn*) et 617 images montrant les yeux fermés (*closed*).
- **Intérêt** (*noyawn + open eyes*) : 1 233 images issues de 616 images de personnes sans bâillement (*noyawn*) et 617 images montrant les yeux ouverts (*open*).

Ces données sont particulièrement adaptées pour détecter des états de vigilance et de somnolence, mais elles seront utiles pour détecter la fatigue chez les étudiants.

Approche pour la collecte et la fusion des données

Afin d'obtenir une base de données cohérente et adaptée à notre modèle, nous avons combiné les deux bases de données en regroupant les catégories similaires et en attribuant des labels homogènes. Les catégories finales et leurs effectifs sont les suivants :

- **Neutralité** (*neutral*) : 5 000 images.
- **Enthousiasme** (*happy*) : 5 000 images.
- **Pas d'envie** (*sad + contempt*) : 5 000 (*sad*) + 3 750 (*contempt*) = 8 750 images.
- **Surprise** (*surprise*) : 5 000 images.
- **Fatigue** (*yawn + closed*) : 1 243 images.
- **Intérêt** (*noyawn + open*) : 1 233 images.

Structure finale de la base de données

Émotion	Nombre d'images	Description
Neutralité	5 000	Expression neutre
Enthousiasme	5 000	Visages heureux
Pas d'envie	5000	Tristesse ou mépris
Surprise	5 000	Expressions surprises
Fatigue	1 243	Bâillements ou yeux fermés
Intérêt	1 233	Absence de bâillement, yeux ouverts

Cette organisation assure une meilleure représentativité des émotions humaines dans différents contextes, en particulier dans les environnements académiques. Ces données sont ensuite prétraitées (normalisation, redimensionnement) pour être utilisées efficacement dans l'entraînement du modèle.

- **Préparation des données :**

Le prétraitement des données est une étape essentielle pour garantir la qualité des données utilisées dans l'entraînement du modèle. Dans ce projet, les données sont d'abord vérifiées pour s'assurer qu'elles sont équilibrées. Cependant, dans certains cas, nous devons appliquer une technique de **data augmentation** pour générer davantage d'images et ainsi améliorer la diversité des données, ce qui permet d'augmenter la robustesse du modèle.

- **Équilibrage des données et utilisation de la Data Augmentation**

Tout d'abord, il est important de noter que la base de données initiale présente un certain déséquilibre entre les différentes classes d'émotions. Par exemple, certaines catégories comme *Fatigue* et *Intérêt* comptent moins d'images que des catégories comme *Neutralité* et *Enthousiasme*. Pour pallier cette problématique et garantir que le modèle ne soit pas biaisé, nous avons utilisé la technique de **data augmentation**.

La **data augmentation** consiste à générer de nouvelles images à partir des images existantes en appliquant des transformations aléatoires. Cela permet de créer un jeu de données plus vaste et plus varié, tout en préservant l'information contenue dans les images d'origine.

Nous avons utilisé le **ImageDataGenerator** de Keras pour appliquer les transformations suivantes :

- **Rotation aléatoire** des images dans une plage de -10 à +10 degrés.
- **Translation horizontale et verticale** aléatoire des images dans une plage de 20% de la taille de l'image.
- **Cisaillement** (shear), qui permet de déformer l'image selon un angle aléatoire.
- **Zoom aléatoire** dans les images.
- **Retournement horizontal** des images.
- **Remplissage des pixels manquants** en utilisant le mode 'nearest' (remplissage par la couleur la plus proche).

Le code pour appliquer ces transformations est le suivant :

```
# Créer un générateur d'images pour l'augmentation des données
datagen_aug = ImageDataGenerator(
    rescale=1./255, |
    rotation_range=10,
    width_shift_range=0.2,
    height_shift_range=0.2,
    shear_range=0.2,
    zoom_range=0.2,
    horizontal_flip=True,
    fill_mode='nearest'
)
```

Afin de rendre les classes équilibrées, nous avons défini le **target_count** comme étant 5 000, ce qui correspond au nombre d'images par classe que nous souhaitons atteindre. Une fois la data augmentation appliquée, nous avons généré des images supplémentaires pour les classes ayant un nombre d'images inférieur à ce seuil.

- **Renommage des fichiers augmentés et conversion en format .jpg**

Après avoir appliqué la data augmentation, les nouvelles images générées sont souvent dans des formats différents, notamment des fichiers au format **.jpeg**. Afin d'assurer la cohérence du format des images dans le jeu de données, nous avons créé un script pour renommer tous les fichiers **.jpeg** en **.jpg**. Ce processus permet de standardiser le format des images pour une gestion plus facile.

Voici le code utilisé pour renommer les fichiers :


```

import os

# Fonction pour renommer les fichiers d'images de .jpeg à .jpg dans le dossier spécifié
def rename_images_in_directory(directory):
    for filename in os.listdir(directory):
        if filename.endswith('.jpeg'):
            # Nouveau nom de fichier avec l'extension .jpg
            new_filename = filename.replace('.jpeg', '.jpg')
            old_filepath = os.path.join(directory, filename)
            new_filepath = os.path.join(directory, new_filename)
            # Renommer le fichier
            os.rename(old_filepath, new_filepath)
            print(f"Renommé {filename} en {new_filename}")

# Dossiers contenant les images augmentées pour chaque classe
classes = ['fatigue', 'Interest', 'contempt']

# Parcourir chaque classe et renommer les fichiers dans le dossier 'augmented'
for class_name in classes:
    augmented_dir = os.path.join(dataset_dir, class_name, 'augmented')
    if os.path.exists(augmented_dir):
        rename_images_in_directory(augmented_dir)
    else:
        print(f"Dossier augmenté non trouvé pour la classe {class_name}: {augmented_dir}")

```

- **Enregistrement des données augmentées**

Une fois les images renommées et formatées, nous avons procédé à leur enregistrement dans les répertoires appropriés pour chaque classe d'émotion. Ces images augmentées sont désormais prêtes à être utilisées dans l'entraînement du modèle, garantissant ainsi une meilleure qualité et diversité des données.

En résumé, le processus de prétraitement a permis de :

1. Équilibrer les données en générant des images supplémentaires via la data augmentation.
2. Normaliser le format des images pour une gestion optimale du jeu de données.
3. Garantir la cohérence et la diversité des données utilisées pour l'entraînement du modèle de reconnaissance des émotions.

- **Gestion des classes d'émotions :**

Une étape essentielle dans la préparation des données pour l'entraînement d'un modèle de machine learning est de diviser le jeu de données en trois ensembles distincts : **Entraînement**, **Validation**, et **Test**. Cette division permet de s'assurer que le modèle est

capable de généraliser ses prédictions à de nouvelles données et d'éviter le surapprentissage (overfitting).

Dans ce projet, après avoir appliqué le processus de **data augmentation** et normalisé les images, nous avons divisé les données en trois ensembles distincts : l'ensemble d'entraînement, l'ensemble de validation, et l'ensemble de test.

1. Entraînement (Train)

L'ensemble d'entraînement contient la majorité des données et est utilisé pour entraîner le modèle. Pour garantir que le modèle soit bien alimenté et que chaque classe soit représentée de manière équilibrée, nous avons créé un dossier **Final_DataSet**, qui contient **5 000 images par classe**. Ces images sont issues de la combinaison des images originales et augmentées, permettant ainsi de générer un jeu de données suffisamment large pour l'entraînement. Cela permet au modèle d'apprendre les différentes variations des émotions exprimées par les étudiants tout en ayant une représentation équilibrée de chaque classe.

2. Validation (Validation)

L'ensemble de validation est utilisé pour évaluer le modèle pendant l'entraînement, permettant ainsi de surveiller ses performances et d'ajuster les hyperparamètres afin d'éviter le surapprentissage. Dans ce projet, nous avons réservé un sous-ensemble de **107 images par classe** pour l'ensemble de validation, ce qui nous permet de disposer d'un nombre suffisant d'exemples pour évaluer le modèle sans pour autant compromettre la taille de l'ensemble d'entraînement.

Cet ensemble de validation, contenu dans le dossier **Final_DataSet_Val**, est crucial pour effectuer des ajustements pendant l'entraînement (par exemple, la régularisation ou le réglage du taux d'apprentissage) tout en préservant la capacité du modèle à généraliser.

3. Test (Test)

L'ensemble de test est utilisé pour évaluer les performances finales du modèle, une fois l'entraînement terminé. Les données de test ne sont pas utilisées pendant l'entraînement ni la validation, ce qui garantit une évaluation objective du modèle sur des données qu'il n'a jamais vues auparavant. Dans notre cas, l'ensemble de test est également

composé de **107 images par classe**, et ces images sont stockées dans le dossier ***Final_DataSet_Test***.

Ces 107 images par classe dans l'ensemble de test permettent de mesurer la précision finale du modèle et d'évaluer sa capacité à prédire correctement les émotions des étudiants sur des données jamais utilisées pendant l'entraînement.

Structure finale des dossiers

- **Dossier *Final_DataSet* (Entraînement) :**

Contient 5 000 images par classe, générées à partir des données originales et augmentées. Ce jeu de données est utilisé pour entraîner le modèle.

- **Dossier *Final_DataSet_Val* (Validation) :**

Contient 107 images par classe. Utilisé pour ajuster les paramètres et évaluer les performances pendant l'entraînement.

- **Dossier *Final_DataSet_Test* (Test) :**

Contient 107 images par classe. Utilisé pour tester le modèle sur de nouvelles données et évaluer ses performances finales.

Cette organisation permet de garantir que les données sont bien réparties, que chaque classe d'émotion est correctement représentée dans chaque ensemble, et que le modèle peut être entraîné efficacement tout en ayant une évaluation précise de sa performance. Ainsi, le modèle est entraîné sur un grand nombre d'exemples, validé de manière adéquate et testé sur un ensemble indépendant pour vérifier sa capacité à généraliser.

4. Entraînement du modèle d'émotions

★ Prétraitement des données et augmentation

L'augmentation des données est utilisée pour enrichir le jeu de données d'entraînement, ce qui permet de réduire le surapprentissage (overfitting) et d'améliorer la généralisation du modèle. Les transformations appliquées incluent :

- **Rotation aléatoire** jusqu'à 30 degrés
- **Décalage horizontal et vertical** pour rendre le modèle plus robuste aux décalages de position
- **Shear** (cisaillement) pour créer une légère distorsion des images
- **Zoom** aléatoire pour simuler des variations de distance de la caméra
- **Retour horizontal** pour créer des variations supplémentaires des images
- **Remplissage des pixels manquants** avec la méthode **'nearest'** lors des transformations géométriques

Les données de validation et de test, quant à elles, sont simplement normalisées (redimensionnées et ré échelonnées) pour garantir la cohérence du prétraitement.

```
# Data Augmentation et Prétraitement
train_datagen = ImageDataGenerator(
    rescale=1./255,
    rotation_range=30,
    width_shift_range=0.2,
    height_shift_range=0.2,
    shear_range=0.2,
    zoom_range=0.2,
    horizontal_flip=True,
    fill_mode='nearest'
)

val_test_datagen = ImageDataGenerator(rescale=1./255)

# Chargement des données
train_data = train_datagen.flow_from_directory(
    train_dir, target_size=(128, 128), batch_size=32, class_mode='categorical'
)

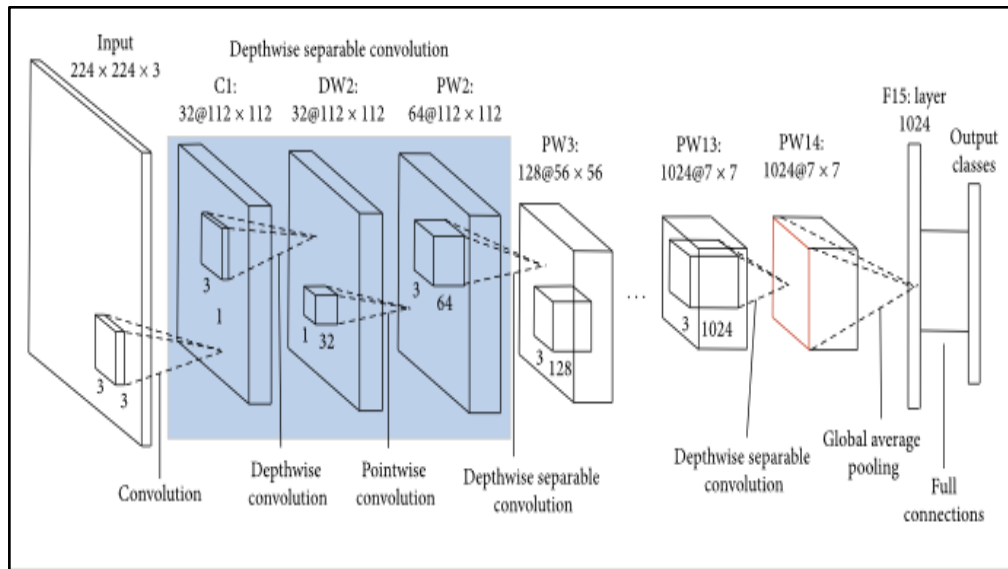
val_data = val_test_datagen.flow_from_directory(
    val_dir, target_size=(128, 128), batch_size=32, class_mode='categorical'
)

test_data = val_test_datagen.flow_from_directory(
    test_dir, target_size=(128, 128), batch_size=32, class_mode='categorical', shuffle=False
)
```

★ Base MobileNet pré-entraînée

MobileNet est un modèle de réseau neuronal convolutif (CNN) pré-entraîné sur ImageNet, ce qui lui permet d'exploiter les connaissances acquises sur un large éventail de classes d'images. MobileNet est connu pour son efficacité en termes de calcul, ce qui est essentiel pour les applications avec des contraintes de ressources ou des dispositifs mobiles. Cette base sert de **feature extractor**. Le modèle pré-entraîné n'inclut pas la

couche de classification finale, car l'objectif est d'ajouter des couches personnalisées adaptées au problème spécifique.



L'architecture MobileNet utilise des **convolutions séparables en profondeur** (depthwise separable convolutions), ce qui signifie que les convolutions standard sont divisées en deux étapes :

1. Une convolution qui traite chaque canal séparément (convolution de profondeur).
2. Une convolution qui effectue une combinaison linéaire des résultats (convolution pointwise).

Cela réduit considérablement le nombre de paramètres et améliore l'efficacité computationnelle, tout en maintenant des performances relativement élevées pour des tâches de classification d'images.

★ Congélation des couches de base

Une fois MobileNet chargé, toutes les couches de la base sont "gelées" (non entraînaibles) dans un premier temps. Cela signifie que les poids des couches de base ne seront pas modifiés pendant l'entraînement initial, ce qui permet de conserver les caractéristiques extraites par le modèle pré-entraîné, tout en évitant de surcharger le modèle avec des ajustements inutiles.

★ Couches personnalisées

Après avoir extrait les caractéristiques avec la base MobileNet, on a ajouter des couches personnalisées pour adapter le modèle à la tâche de classification :

- **GlobalAveragePooling2D** : Cette couche réduit les dimensions des cartes de caractéristiques en effectuant une moyenne sur chaque carte de caractéristiques. Cela permet de réduire considérablement le nombre de paramètres, tout en conservant des informations pertinentes. L'output de cette couche est un vecteur de taille fixe qui résume les informations de toutes les cartes de caractéristiques.
- **Dense (128 unités, ReLU)** : Une couche entièrement connectée avec 128 neurones et une fonction d'activation **ReLU** (Rectified Linear Unit), qui permet au modèle de capturer des relations complexes entre les caractéristiques extraites. La régularisation L2 est utilisée ici pour pénaliser les grands poids, ce qui aide à prévenir l'overfitting.
- **Dropout (0.5)** : Cette technique est utilisée pour régulariser le modèle en désactivant de manière aléatoire 50% des neurones pendant l'entraînement. Cela aide à réduire l'overfitting en forçant le modèle à apprendre des représentations plus robustes.
- **Dense (niveaux de classes, Softmax)** : La couche finale du modèle est une couche dense avec une activation **Softmax**, qui génère une probabilité pour chaque classe dans la tâche de classification. Le nombre de neurones dans cette couche correspond au nombre de classes qui est 6 de notre jeu de données d'entraînement (comme spécifié par `train_data.num_classes`).

★ Fine-Tuning (déverrouillage de couches)

Après une phase d'entraînement initiale avec les couches de base gelées, on va effectuer un **fine-tuning** en dégelant certaines des dernières couches de la base MobileNet. Cela permet au modèle d'ajuster les caractéristiques extraites par MobileNet pour mieux correspondre à notre problème spécifique. En dégelant seulement les 50 dernières couches, ce qui permet au modèle de continuer à apprendre des caractéristiques plus spécifiques, sans perdre les connaissances générales acquises par la partie pré-entraînée.

```

# Chargement du modèle MobileNet (pré-entraîné)
base_model = MobileNet(weights='imagenet', include_top=False, input_shape=(128, 128, 3))

# Geler les couches de base
for layer in base_model.layers[:-50]: # Déverrouiller les 50 dernières couches
    layer.trainable = True

# Ajout des couches personnalisées
x = base_model.output
x = GlobalAveragePooling2D()(x)
x = BatchNormalization()(x) # Normalisation des données
x = Dense(256, activation='relu', kernel_regularizer=l2(0.01))(x) # Couche dense avec régularisation L2
x = Dropout(0.5)(x) # Dropout pour éviter l'overfitting
x = Dense(128, activation='relu', kernel_regularizer=l2(0.01))(x)
x = Dropout(0.5)(x)
output = Dense(6, activation='softmax')(x) # 6 classes à prédire

# Création du modèle
model = Model(inputs=base_model.input, outputs=output)

```

★ Compilation du modèle

Le modèle est compilé avec l'optimiseur **Adam** (un optimiseur adaptatif très performant), avec un taux d'apprentissage faible (1e-4) pour éviter des ajustements trop importants des poids durant l'entraînement. La fonction de perte utilisée est **categorical_crossentropy**, adaptée à un problème de classification multiclasse, et l'évaluation de la performance est réalisée avec la **précision (accuracy)**.

★ Régularisation et prévention de l'overfitting

Des techniques telles que **early stopping** et **reduce learning rate on plateau** sont utilisées pour éviter l'overfitting :

- **Early stopping** arrête l'entraînement si la perte sur les données de validation cesse de s'améliorer pendant un certain nombre d'époques (défini par **patience=5**), ce qui empêche l'entraînement excessif du modèle.
- **Reduce learning rate on plateau** réduit automatiquement le taux d'apprentissage lorsque la perte sur la validation cesse d'améliorer, permettant au modèle de continuer à affiner ses poids tout en évitant des sauts trop importants dans l'espace des paramètres.

```
# Compilation du modèle avec un taux d'apprentissage réduit
optimizer = tf.keras.optimizers.Adam(learning_rate=5e-5) # Taux d'apprentissage très bas
model.compile(optimizer=optimizer,
              loss='categorical_crossentropy',
              metrics=['accuracy'])

# Callbacks pour éviter l'overfitting
early_stopping = EarlyStopping(monitor='val_loss', patience=7, restore_best_weights=True)
reduce_lr = ReduceLRonPlateau(monitor='val_loss', factor=0.2, patience=4, min_lr=1e-7)
```

★ Entraînement et évaluation

L'entraînement du modèle s'effectue sur les données d'entraînement avec des données de validation pour évaluer la performance en temps réel. Le modèle est ensuite évalué sur un jeu de test séparé pour mesurer sa capacité à généraliser sur de nouvelles données.

```
class_weights = class_weight.compute_class_weight(
    'balanced', classes=np.unique(train_data.labels), y=train_data.labels
)
class_weights_dict = dict(enumerate(class_weights))
def scheduler(epoch, lr):
    if epoch < 10:
        return lr
    else:
        return lr * 0.95 # Réduit le taux d'apprentissage de 5% après chaque époque

lr_scheduler = tf.keras.callbacks.LearningRateScheduler(scheduler)

model.fit(train_data, validation_data=val_data, epochs=20, callbacks=[early_stopping, reduce_lr, lr_scheduler], verbose=1)
```

★ Sauvegarde et visualisation des résultats

À la fin de l'entraînement, le modèle est sauvegardé pour être utilisé ultérieurement. Les courbes de **précision** et de **perte** sont tracées pour visualiser l'évolution de la performance du modèle au fil des époques, ce qui permet d'analyser l'efficacité de l'entraînement et de vérifier si le modèle souffre d'overfitting. Pour notre modèle, nous avons trouvé:

Test Loss: 0.6066

Test Accuracy: 0.8053

5. Reconnaissance Faciale

★ Technologies Utilisées

La reconnaissance faciale repose sur la bibliothèque **face_recognition**, permettant d'extraire des encodages précis des visages pour les différencier. Les fonctionnalités principales incluent :

- **Encodage des visages** : Transformation des images des étudiants en représentations numériques uniques.
- **Comparaison des visages** : Détection des correspondances entre les visages détectés en temps réel et ceux stockés dans la base de données via api **Facial Recognition**.
- **Gestion des cas inconnus** : Identification des visages qui ne correspondent pas à ceux déjà enregistrés.

★ Gestion des Visages Inconnus et Connus

- Les visages détectés sont comparés avec les encodages de la base de données pour retrouver l'identité des étudiants.
- Si une correspondance est trouvée, le système attribue un nom à l'étudiant ; sinon, le visage est marqué comme **"inconnu"**.

6. Développement de l'Interface Utilisateur

★ Choix de l'Interface

L'interface utilisateur a été développée avec **Streamlit**, une bibliothèque Python légère et rapide pour créer des applications interactives.

Streamlit a été choisi pour sa simplicité d'intégration avec des bibliothèques de machine learning et ses fonctionnalités pour le suivi en temps réel.

★ Fonctionnalités Principales

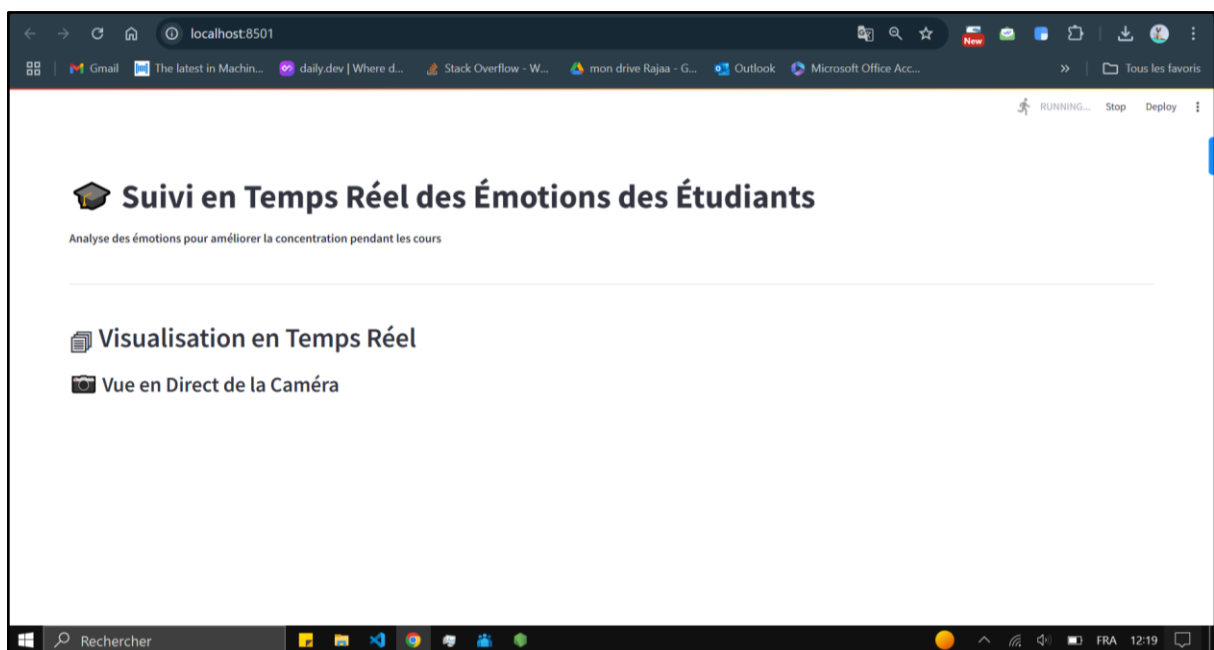
- **Suivi en temps réel** : Visualisation des émotions captées par le modèle pour chaque étudiant.
- **Graphiques dynamiques** : Répartition des émotions globales, individuelles et leur évolution dans le temps.
- **Alertes en cas de baisse de concentration** : Notifications affichées en temps réel pour signaler des seuils critiques.

- Intégration de graphiques "**donut**" pour représenter les émotions globales en pourcentages.
- Suivi détaillé de l'évolution des émotions pour chaque étudiant, permettant d'observer les variations au cours du temps.

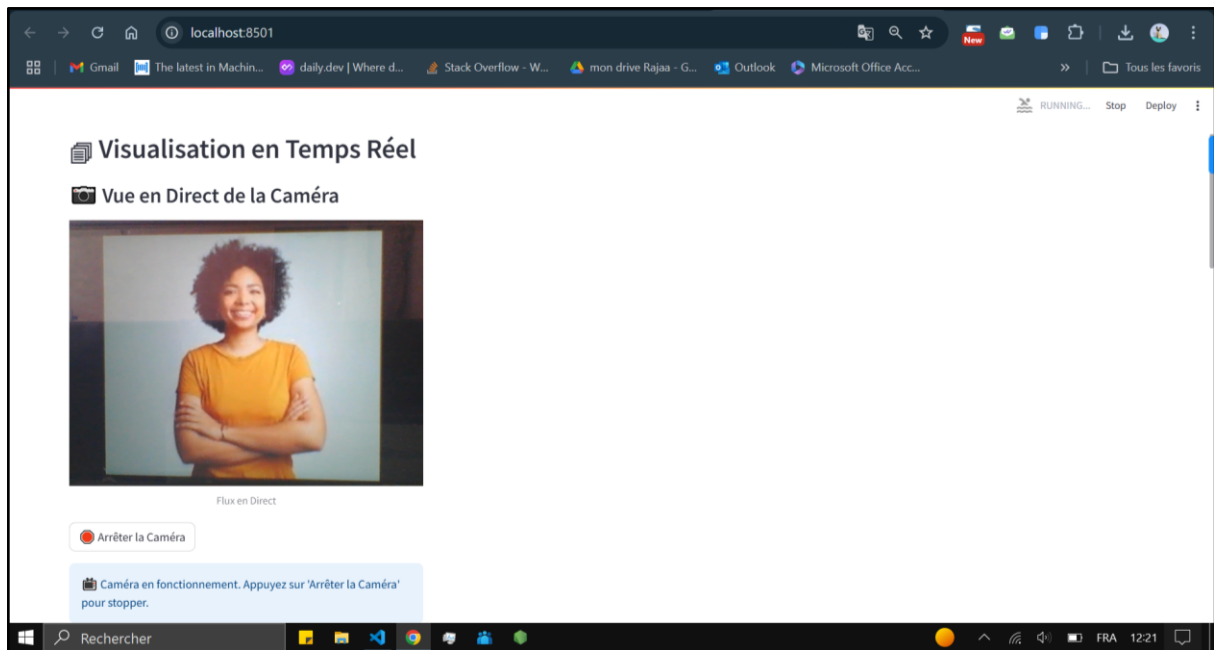
❖ Capture d'Écran de l'Interface

Des captures d'écran ont été ajoutées pour illustrer les différentes sections de l'application :

1. Interface principale



La première interface qui s'affiche est celle-là. Ici la caméra en direct est en cours de chargement.



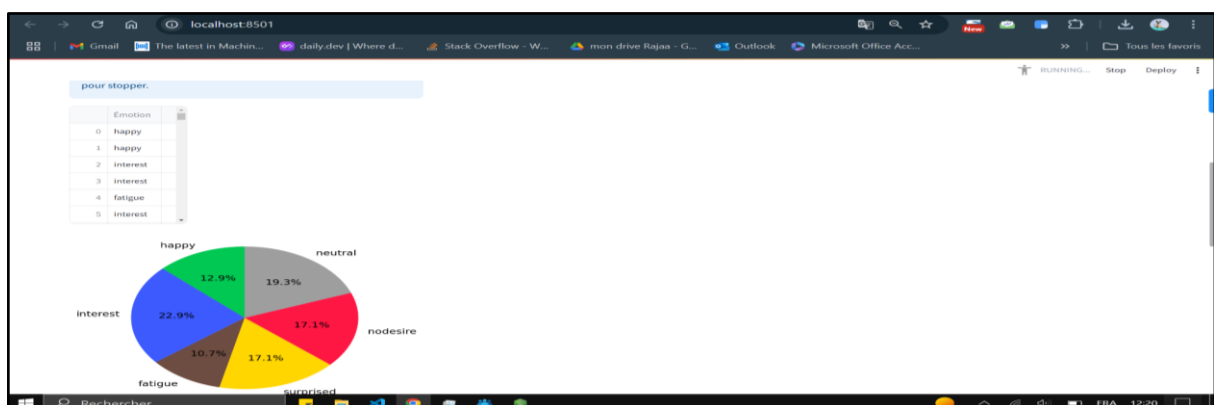
Ensuite, on voit que la caméra est chargée et elle est déclenchée en mode direct(*streaming*).

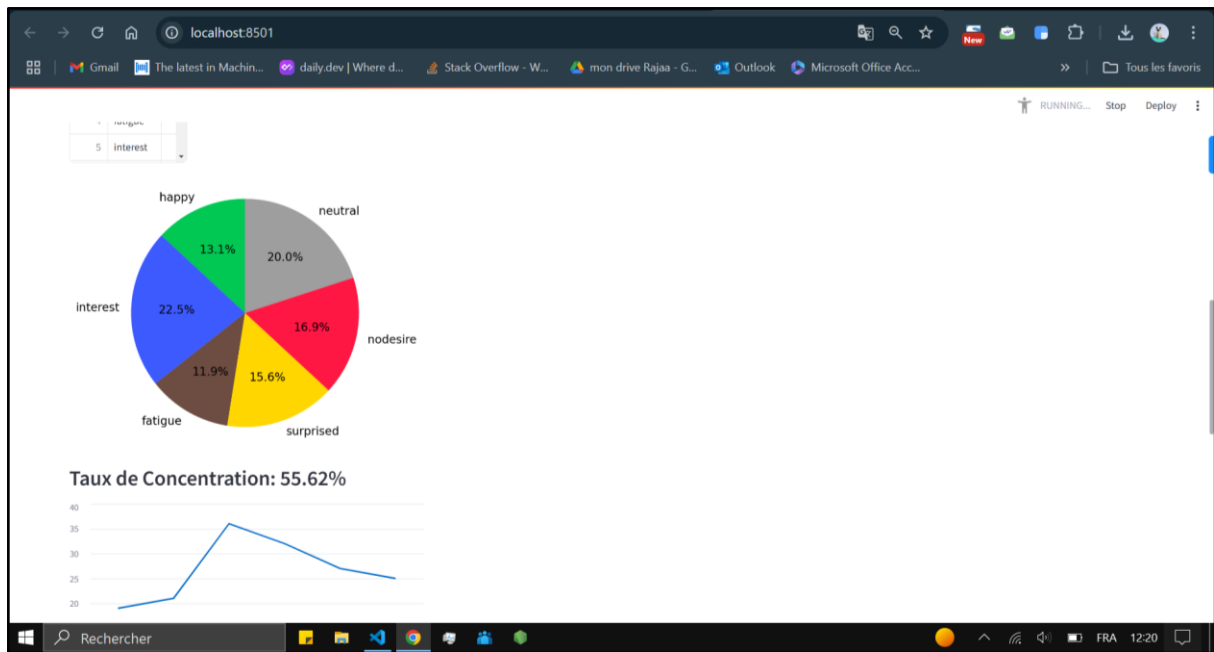
2. Courbes d'évolution des émotions des étudiants.

Une comparaison a été effectuée pour analyser les émotions détectées chez chaque étudiant. Cette analyse met en évidence les tendances individuelles, telles qu'un niveau de **fatigue** accru chez certains étudiants par rapport à d'autres.

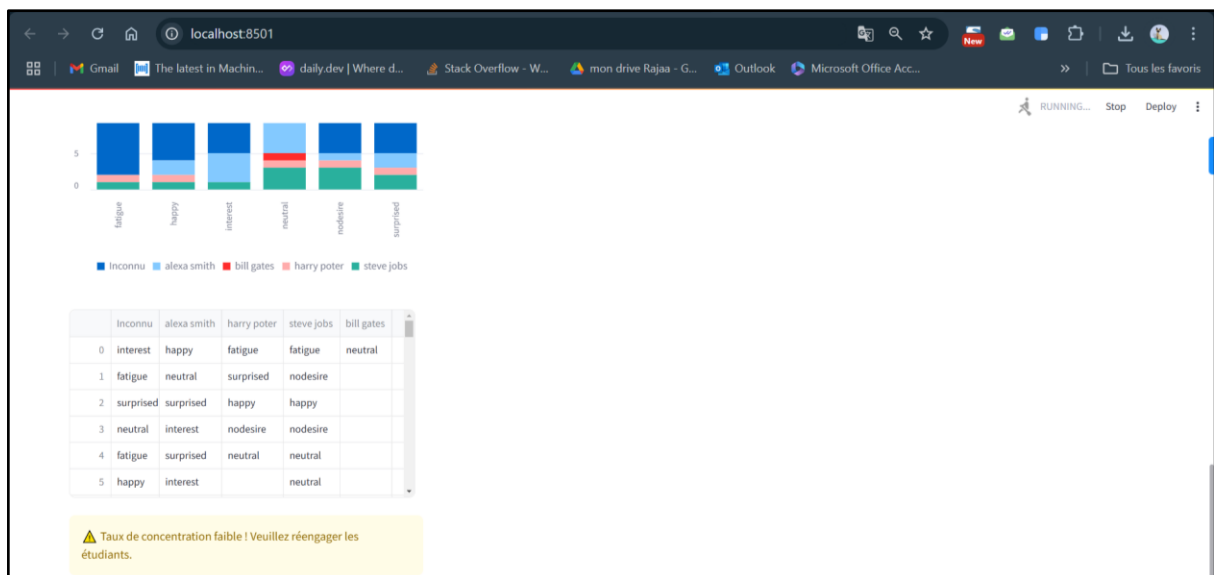
Les émotions sont suivies en temps réel, et leur évolution est présentée sous forme de courbes cumulées. Cela permet d'identifier :

- Les moments où les étudiants montrent des **signes de concentration** (émotions comme *neutral* et *interest*).
- Les périodes où la **fatigue** ou le **désintérêt** augmentent.





3. Alertes de concentration affichées en direct.



Si le taux de concentration est minimum, on affiche un message d'alerte.

★ Matrice de confusion et limites du Système

La matrice de confusion ci-dessus met en évidence certaines limites du modèle de reconnaissance des émotions :

1. Confusions entre Classes Similaires :

- Les émotions **"happy"** et **"neutre"** présentent une confusion notable. Par exemple, 22 prédictions étiquetées comme "neutre" sont en réalité "happy". Cela peut s'expliquer par des similitudes subtiles dans les expressions faciales associées à ces émotions.

2. Faible Précision pour Certaines Classes :

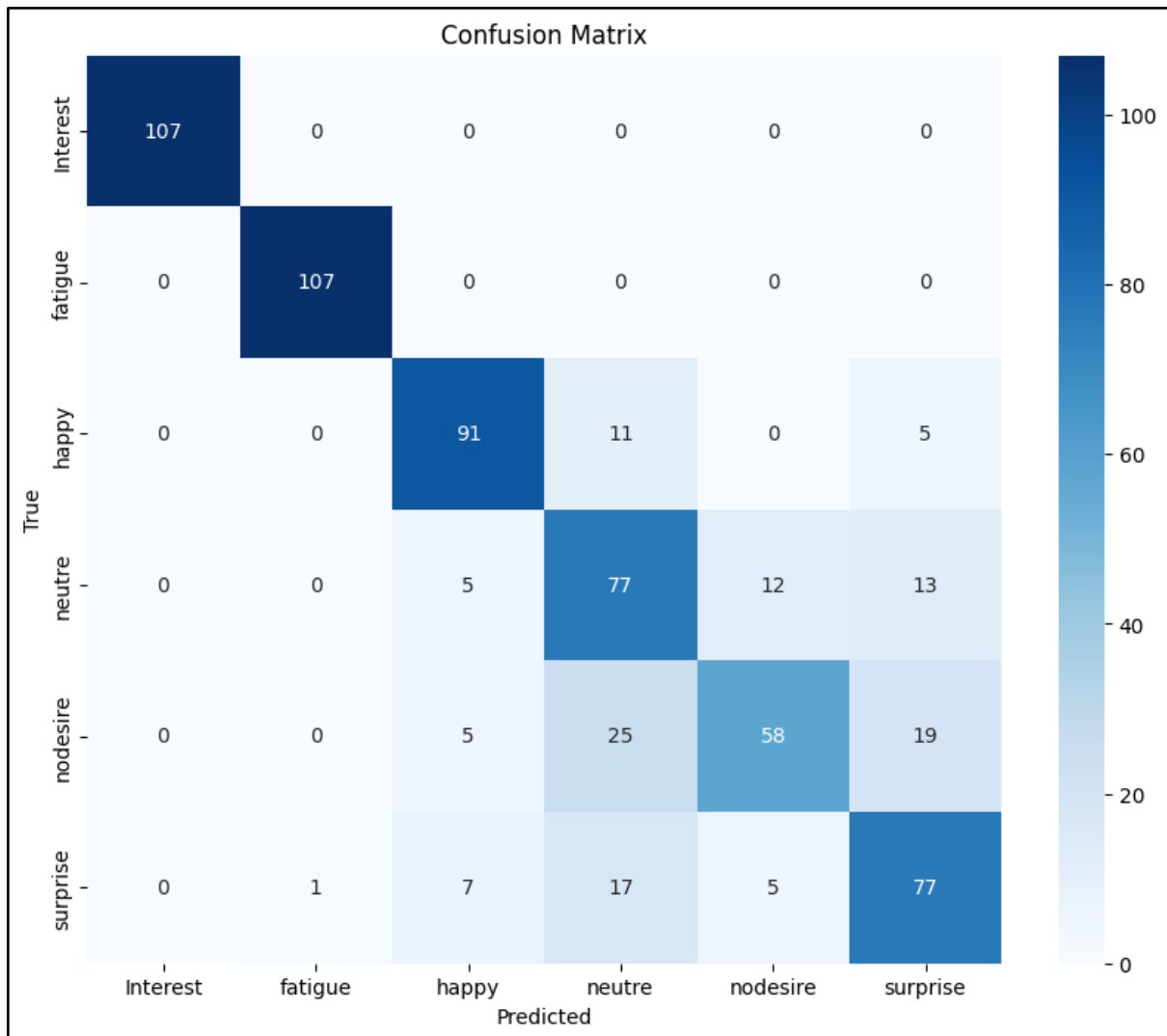
- Les classes comme **"contempt"** et **"neutre"** montrent des niveaux de confusion importants. Par exemple :
 - "Contempt" est souvent prédit comme "happy" ou "sad", ce qui réduit la précision globale.
 - Les prédictions incorrectes pour "neutre" incluent des étiquettes "happy" (19) et "sad" (30).

3. Bonne Détection pour Certaines Émotions :

- Les classes **"interest"** et **"fatigue"** sont bien détectées, comme le montrent leurs valeurs diagonales élevées (106 et 107, respectivement). Cela peut refléter des différences claires dans les caractéristiques visuelles de ces émotions par rapport aux autres.

4. Confusion Marginale avec "surprise" :

- L'émotion "surprise" est parfois confondue avec "happy" (11 cas) ou "neutre" (18 cas), ce qui peut s'expliquer par des expressions partagées comme des sourires ou des changements soudains dans les traits du visage.



8. Perspectives et améliorations

Pour surmonter ces limites, les améliorations suivantes sont suggérées :

- **Augmentation de Données** : Collecter davantage d'exemples pour les classes minoritaires comme "contempt" ou "neutre" ou pour d'autres émotions.
- **Affinage des caractéristiques** : Exploiter des méthodes avancées d'extraction de caractéristiques pour distinguer les émotions similaires.
- **Détection de tricherie**: il est possible de rechercher des bases de données contenant des exemples de comportements associés à la tricherie, comme des images ou des vidéos de surveillance en salle d'examen. En utilisant ces données, un modèle d'apprentissage automatique ou d'intelligence artificielle peut être entraîné pour identifier des signes de tricherie, tels que des gestes suspects, des mouvements inhabituels ou des objets non autorisés. Par exemple, un modèle de vision par ordinateur peut analyser des vidéos en temps réel pour repérer des

comportements tels que l'extraction de documents de la poche ou l'utilisation de dispositifs électroniques.

9. Conclusion

Ce projet a permis d'explorer le potentiel des techniques d'apprentissage automatique pour résoudre un problème complexe : la reconnaissance et la classification des émotions. À travers les différentes étapes du projet, de la préparation des données à l'évaluation des performances du modèle, nous avons démontré la puissance et les limites des approches basées sur les réseaux neuronaux convolutifs. Les résultats obtenus montrent une bonne capacité de reconnaissance pour certaines classes, mais révèlent également des confusions importantes pour d'autres, soulignant la nécessité d'un travail approfondi sur des aspects tels que l'équilibrage des données et le choix des hyperparamètres.

En plus des défis techniques, ce projet a mis en lumière l'importance d'une méthodologie rigoureuse pour garantir la fiabilité des résultats, ainsi que la pertinence de l'interprétation des sorties du modèle. Les visualisations telles que la matrice de confusion ont joué un rôle clé dans la compréhension des forces et faiblesses du système.

Malgré certaines limites, ce projet constitue une base solide pour des travaux futurs. Il ouvre la voie à plusieurs pistes d'amélioration, notamment l'utilisation de données multi-modales (audio, vidéo et texte), l'intégration de techniques avancées telles que les modèles transformateurs, et l'exploration d'applications concrètes dans des domaines variés comme la santé, l'éducation, et les systèmes interactifs. Ce travail illustre non seulement l'intérêt croissant pour l'intelligence artificielle dans la reconnaissance des émotions, mais également les défis à relever pour en faire une technologie véritablement performante et accessible.

Contact :

rajaa.lebnaiti@gmail.com [Détection des émotions des étudiants lors des cours en ligne]